

category: python-tkinter

tags: [python, tkinter, startup-performance, lazy-import, torch, paddleocr]

severity: high

created: 2026-07-26

updated: 2026-07-26

project-origin: KZTEK Image Tools (d:\Tool)

App Python khởi động chậm 40s+ vì import thư viện ML ở mức module

Tình huống gặp phải

Audit hiệu năng app Tkinter đa tab (KZTEK Image Tools, 89 file, 44.6k dòng).

User phản nản "mở app lâu". Đo `import tool.core.app` = **41,15 giây** trên

Python 3.10 + torch 2.5.1+cu121.

Triệu chứng / Lỗi

Không có exception — app vẫn chạy đúng, chỉ là cửa sổ mất ~46 giây mới hiện, trông như bị treo. Không có splash screen nên user không biết app đang làm gì.

Nguyên nhân gốc rễ

Hai tầng, và tầng nặng nhất bị bỏ sót nếu chỉ quét regex theo tên thư viện:

1. **Import thư viện nặng ở mức module** — kể cả khi nằm trong `try/except ImportError` để lấy cờ `_X_OK`, dòng import vẫn CHẠY THẬT lúc nạp module:

```
```python
```

```
try:
```

```
from rfdetr import RFDETRBase # 11,5s — chạy dù user không dùng RF-DETR
```

```
_RFDETR_OK = True
```

```
except ImportError:
```

```
_RFDETR_OK = False
```

```
`
```

2. **Import gián tiếp** — thủ phạm lớn nhất, khó thấy nhất.

`core/imports.py` (file được `app.py` import ở dòng đầu) có

`from paddleocr import PaddleOCR.paddleocr` kéo theo `torch` (7,8s) +

`transformers` + `matplotlib`. Grep `import torch` trong toàn repo **\*\*KHÔNG**

ra kết quả nào ở file này\*\* — vì `torch` bị kéo vào gián tiếp.

3. Cộng thêm: `App.__init__` dựng đủ 17 tab rồi mới ẩn tab user đã tắt (+3,9s).

## Giải pháp

Tách "kiểm tra thư viện có cài không" khỏi " nạp thư viện":

```
shared/lazy_import.py
import importlib, importlib.util

def module_available(name: str) -> bool:
 """True nếu module đã cài — KHÔNG import (find_spec chỉ tra sys.path)."""
 try:
 return importlib.util.find_spec(name) is not None
 except (ImportError, ValueError, ModuleNotFoundError):
 return False

class _LazyCallable:
 """Proxy gọi được — chỉ import ở lần dùng đầu tiên."""
 __slots__ = ("_m", "_a")
 def __init__(self, m, a): self._m, self._a = m, a
 def __call__(self, *a, **k):
 return getattr(importlib.import_module(self._m), self._a)(*a, **k)
 def __getattr__(self, n):
 return getattr(getattr(importlib.import_module(self._m), self._a), n)
```

Chỗ gọi giữ nguyên cú pháp cũ:

```
_YOLO_OK = module_available("ultralytics") # thay cờ try/except
YOLO = lazy_callable("ultralytics", "YOLO") # YOLO(path) vẫn viết y như cũ
```

Với `matplotlib` phải gói riêng vì `matplotlib.use("TkAgg")` bắt buộc chạy

TRƯỚC khi import backend:

```
def matplotlib_tk():
 global _cache
 if _cache is None:
 import matplotlib; matplotlib.use("TkAgg")
 from matplotlib.figure import Figure
 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
 _cache = (Figure, FigureCanvasTkAgg)
 return _cache
```

Tkinter: add khung tab RỖNG vào Notebook lúc `__init__`, dựng nội dung ở

handler `<<NotebookTabChanged>>` khi user mở tab lần đầu.

**Kết quả:** 46s → 2,08s. Mở tab lần đầu tốn thêm 0,3–2,0s (đúng lúc user cần).

## Cách truy vết thủ phạm (quan trọng hơn cả giải pháp)

Grep tên thư viện là **chưa đủ** — không thấy import gián tiếp. Phải hook

`builtins.__import__` để in stack lúc thư viện nặng bị nạp lần đầu:

```
import builtins, traceback
_real, _seen = builtins.__import__, set()
def _hook(name, *a, **k):
 top = name.split(".")[0]
 if top in ("torch", "matplotlib", "transformers") and top not in _seen:
 _seen.add(top)
 print("### import", top, "<-- gọi tu:")
```

```
for fr in traceback.extract_stack()[::-1]:
 if "tool" in fr.filename: # lọc file của mình
 print(" %s:%d %s" % (fr.filename, fr.lineno, (fr.line or "").strip()))
 return _real(name, *a, **k)
builtins.__import__ = _hook
import my_app # <-- stack in ra chỉ đúng dòng gây ra
```

## Bẫy cần tránh khi áp dụng

- `find_spec()` chỉ khẳng định module TỒN TẠI. Nếu bản cài hỏng (VD `paddleocr` thiếu `paddlepaddle`), lỗi sẽ dòi từ lúc khởi động sang lúc gọi thật. Nếu code cũ dùng `except Exception` (không phải `except ImportError`) thì phải \*\*thêm try/except ở chỗ khởi tạo\*\* để giữ nguyên thông báo lỗi thân thiện cho user.
- Nếu module A dựa vào module B đã gọi `matplotlib.use("TkAgg")` lúc khởi động, lazy hóa B sẽ làm A lấy nhầm backend. Cho mọi nơi đi chung một helper.
- Đo lại bằng `sys.modules` chứ không chỉ bằng đồng hồ:  
`print("torch" in sys.modules)` sau khi import app — phải là `False`.

## Liên quan

- `[[python-tkinter-settings-debounce]]` — cùng đợt audit, settings ghi mỗi phím gõ